

DAFCOM ML Pipeline Refactoring Plan

This document outlines an incremental plan to refactor the DAFCOM ML pipeline into a production-ready, modular, and scalable codebase. Each step includes a description, example code, and a list of files to modify. The plan emphasizes the use of modern Python class-based structures for modularity and maintainability.

A. Documentation Standards (NumPy-style Docstrings)

Goal: Use NumPy-style docstrings for all public APIs, documenting arguments, return values, exceptions, and side effects. Include example usage for complex functions. Ensure compatibility with MkDocs + makedocstrings.

Rationale: Consistent, comprehensive documentation improves maintainability, onboarding, and enables automated API docs. NumPy-style docstrings are widely adopted and supported by documentation tools.

Example:

```
def add(a, b):  
    """  
    Add two numbers.  
  
    Parameters  
    -----  
    a : int or float  
        First number.  
    b : int or float  
        Second number.  
  
    Returns  
    -----  
    result : int or float  
        The sum of a and b.  
  
    Examples  
    -----  
    >>> add(1, 2)  
    3  
    """  
    return a + b
```

Files to Modify: - All public classes and functions

B. Environment & Dependency Management

Goal: Use `pyproject.toml` (and optionally `requirements.txt`, `environment.yml`, `Dockerfile`) for environment and dependency management.

Rationale: Explicit environment management ensures reproducibility, easier onboarding, and compatibility across systems.

Files to Modify: - Add `pyproject.toml` (and optionally `requirements.txt`, `environment.yml`, `Dockerfile`)

C. Data Validation & Quality Checks

Goal: Add data schema validation, missing value handling, and outlier detection as part of preprocessing.

Rationale: Data quality issues are a major source of pipeline failures. Automated validation and cleaning ensure robust, reliable ML results.

Example:

```
import pandas as pd
from pandera import DataFrameSchema, Column, Check

class DataValidator:
    def __init__(self, schema: DataFrameSchema):
        self.schema = schema
    def validate(self, df: pd.DataFrame) -> pd.DataFrame:
        return self.schema.validate(df)
```

Files to Modify: - Add `datavalidator.py` or integrate into `preprocessor.py`

D. Visualization Class/Module

Goal: Centralize all plotting and reporting in a dedicated Visualization class/module.

Rationale: Centralizing visualization logic improves code reuse, maintainability, and makes it easy to update or extend visualization capabilities.

Example:

```
import matplotlib.pyplot as plt

class Visualization:
    def plot_timeseries(self, df, x_col, y_col, title=None):
        """
        Plot a time series.
```

```

Parameters
-----
df : pandas.DataFrame
    Data to plot.
x_col : str
    Column for x-axis.
y_col : str
    Column for y-axis.
title : str, optional
    Plot title.
"""
plt.figure(figsize=(10, 5))
plt.plot(df[x_col], df[y_col])
plt.title(title or f"{y_col} over {x_col}")
plt.xlabel(x_col)
plt.ylabel(y_col)
plt.show()

```

Files to Modify: - Add visualization.py

E. Pipeline Entry Point & CLI

Goal: Provide a clear entry point (main.py) and a CLI interface (e.g., using argparse or click) for running the pipeline.

Rationale: A single entry point and CLI make the pipeline easy to use, automate, and integrate into other systems.

Example:

```

# main.py
import argparse
from src.forecast_mode.trainer import ModelTrainer

def main(config_path):
    trainer = ModelTrainer(config_path)
    trainer.run()

if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument("--config", required=True)
    args = parser.parse_args()
    main(args.config)

```

Files to Modify: - Add main.py

F. Monitoring, Logging, and Performance Tracking

Goal: Implement robust monitoring of logs, errors, and model performance. Plan for model/data drift detection and alerting in production.

Rationale: Monitoring and alerting are essential for production ML systems to ensure reliability, detect issues early, and maintain model quality.

Example:

```
import logging
import mlflow

class Logger:
    def __init__(self):
        logging.basicConfig(level=logging.INFO)
    def info(self, msg):
        logging.info(msg)
    def error(self, msg):
        logging.error(msg)

class PerformanceTracker:
    def log_metrics(self, metrics: dict):
        mlflow.log_metrics(metrics)
```

Files to Modify: - Extend `logger.py`, add `performancetracker.py` or integrate with MLflow

1. Codebase Audit & Modularization

Goal: Organize scripts into logical modules (io, preprocessing, features, models, evaluation, utils) and begin transitioning to class-based design for each major component.

Rationale: The current codebase consists of many standalone scripts, which can lead to code duplication, inconsistent logic, and difficulty in understanding or extending the workflow. Modularization and class-based design improve code organization, make it easier to maintain and test, and allow for code reuse across different pipeline stages.

Example: - Move all data loading logic into a `DataLoader` class in `dataloader.py`. - Create a `models/` directory for ML code, with classes for each model type.

Files to Modify: - All scripts in `src/forecast_mode/step1_input_prepare/`, `step4_ML_LSTM/`, `step6_ML_Xgboost/`, etc. - Create new files: `src/forecast_mode/dataloader.py`,

src/forecast_mode/models/, etc.

```
def load_data(path: str) -> pd.DataFrame:
```

2. Modern Python Best Practices

Goal: Add type hints, docstrings, logging, and error handling. Set up testing. Use class-based design for modularity.

Rationale: The current scripts may lack consistent documentation, error handling, and logging, making debugging and collaboration difficult. Adopting best practices ensures code clarity, reliability, and easier onboarding for new contributors. Type hints and docstrings improve code readability and enable better tooling support.

Example:

```
import logging
from typing import List

class DataLoader:
    def load(self, path: str) -> pd.DataFrame:
        """Load data from a CSV file."""
        try:
            return pd.read_csv(path)
        except Exception as e:
            logging.error(f"Failed to load {path}: {e}")
            raise
```

Files to Modify: - All Python scripts, especially in src/forecast_mode/ - Add tests/ directory for pytest

```
def read_netcdf(file_path: str) -> xr.Dataset:
def read_csv(file_path: str) -> pd.DataFrame:
```

3. Data IO Refactor (pandas/xarray)

Goal: Use pandas for tabular and xarray for gridded data. Centralize IO logic in dedicated classes.

Rationale: Current scripts may use ad-hoc file operations, which can be error-prone and hard to scale. Centralizing IO logic in classes using pandas and xarray leverages powerful, well-supported libraries for efficient data handling, improves code reuse, and makes it easier to update IO logic in one place.

Example:

```

import xarray as xr
import pandas as pd

class DataLoader:
    def read_netcdf(self, file_path: str) -> xr.Dataset:
        return xr.open_dataset(file_path)

    def read_csv(self, file_path: str) -> pd.DataFrame:
        return pd.read_csv(file_path)

```

Files to Modify: - All scripts that read/write data, e.g., `step1_extract_BEV.py`, `step2_surface_map.py`, etc. - `dataloader.py`

```
df = dd.read_csv('large_file.csv')
```

4. Parallelism & Scalability (dask)

Goal: Integrate dask for parallel and out-of-core processing, encapsulated in processing classes.

Rationale: Script-based data processing can be limited by memory and single-threaded execution. Integrating dask enables scalable, parallel, and out-of-core computation, allowing the pipeline to handle much larger datasets efficiently and future-proofing the system for growth.

Example:

```

import dask.dataframe as dd

class Preprocessor:
    def group_mean(self, file_path: str, col: str):
        df = dd.read_csv(file_path)
        return df.groupby(col).mean().compute()

```

Files to Modify: - Data processing scripts in `step1_input_prepare/`, `step3_interpolate/`, etc.

```
nn = keras.Sequential([...]) nn.compile(...) nn.fit(X_train, y_train)
```

5. ML Pipeline Refactor (XGBoost/Keras)

Goal: Standardize model code using XGBoost and Keras APIs, encapsulated in model classes.

Rationale: Current ML code may be scattered across scripts, making it hard to maintain, extend, or swap models. Encapsulating ML logic in classes stan-

standardizes interfaces, simplifies experimentation, and supports best practices for reproducibility and deployment.

Example:

```
from xgboost import XGBRegressor
from tensorflow import keras

class XGBModel:
    def __init__(self, **params):
        self.model = XGBRegressor(**params)
    def train(self, X, y):
        self.model.fit(X, y)

class LSTMModel:
    def __init__(self, layers, **compile_args):
        self.model = keras.Sequential(layers)
        self.model.compile(**compile_args)
    def train(self, X, y):
        self.model.fit(X, y)
```

Files to Modify: - step4_ML_LSTM/step2_LSTM_model_build.py -
step6_ML_Xgboost/step3_xgboost_training.py

6. Configuration Management (YAML)

Goal: Use a master YAML file for all config, loaded and managed by a configuration class.

Rationale: Hardcoded parameters in scripts make the pipeline inflexible and error-prone. Centralizing configuration in a YAML file, managed by a class, enables easy tuning, reproducibility, and clear separation between code and settings.

Example:

```
# config.yaml
input_path: data/input/
model_params:
    learning_rate: 0.01

import yaml
class Config:
    def __init__(self, config_path):
        with open(config_path) as f:
            self.config = yaml.safe_load(f)
    def get(self, key, default=None):
        return self.config.get(key, default)
```

Files to Modify: - All scripts (replace hardcoded params) - Add `config.yaml`

7. Continual Learning Support

Goal: Enable incremental model updates and checkpointing, managed by model classes.

Rationale: Script-based training often requires retraining from scratch. Supporting continual learning and checkpointing allows the system to adapt to new data, reduces compute costs, and improves model performance over time.

Example:

```
class XGBModel:
    def train(self, X, y, previous_model=None):
        self.model.fit(X, y, xgb_model=previous_model)
```

Files to Modify: - Model training scripts

8. Output & Logging Control

Goal: Standardize and limit output, use logging for status, and encapsulate logging in classes where appropriate.

Rationale: Uncontrolled print statements and inconsistent output make it hard to monitor and debug the pipeline. Centralized logging provides clear, configurable, and consistent status reporting, which is essential for production systems.

Example:

```
import logging
class Logger:
    def __init__(self):
        logging.basicConfig(level=logging.INFO)
    def info(self, msg):
        logging.info(msg)
Logger().info('Processing complete.')
```

Files to Modify: - All scripts

9. Production Hardening

Goal: Add tests, docs, and CI/CD. Write unit tests for each class and method.

Rationale: Script-based workflows are rarely tested, leading to undetected bugs and regressions. Adding tests, documentation, and CI/CD ensures code quality, reliability, and smooth collaboration, and supports sustainable long-term development.

Example: - Add `tests/` with pytest tests for each class (e.g., `TestDataLoader`, `TestPreprocessor`, `TestXGBModel`). - Add `docs/` with usage instructions and class documentation. - Set up GitHub Actions for CI.

Files to Modify: - Add `tests/`, `docs/`, `.github/workflows/ci.yml`

10. Adopt Modern Python Class Structure

Goal: Encapsulate all major pipeline components (data IO, preprocessing, modeling, configuration, logging) into Python classes for modularity, reusability, and maintainability.

Rationale: The current script-based approach makes it difficult to extend, test, or reuse code. Adopting a class-based structure brings the benefits of object-oriented programming: encapsulation, inheritance, and clear interfaces, which are essential for building robust, production-grade systems.

Example:

```
class DataLoader:
    def __init__(self, config):
        self.config = config
    def load(self):
        # implement data loading logic
        pass

class Preprocessor:
    def __init__(self, config):
        self.config = config
    def transform(self, data):
        # implement preprocessing logic
        pass

class ModelTrainer:
    def __init__(self, config):
        self.config = config
    def train(self, X, y):
        # implement training logic
        pass
```

Files to Modify: - Refactor scripts in `src/forecast_mode/` (e.g., `step1_input_prepare/`, `step4_ML_LSTM/`, `step6_ML_Xgboost/`) to use

class-based design. - Create new modules: `dataloader.py`, `preprocessor.py`, `trainer.py`, etc.

This plan will incrementally transform DAFCOM into a robust, scalable, and maintainable ML pipeline. Each step should be validated before proceeding to the next.